

01_SYSTEM.md

SpeechStudio System Specification

Version: 1.0

Status: Approved

1. Purpose

This document defines the overall system architecture and runtime behaviour of SpeechStudio.

It specifies how the application starts, initializes its environment, manages resources and communicates internally.

The document intentionally excludes user interface implementation details.

2. System Overview

SpeechStudio is a portable Windows desktop application for local speech synthesis using Higgs Audio V3.

The application shall operate without requiring manual Python configuration or command line interaction.

The application shall remain fully functional after the initial setup without requiring an internet connection.

3. System Components

SpeechStudio consists of the following logical components.

Launcher

Bootstrap

User Interface

Speech Engine

Prompt Builder

Voice Manager

Model Manager

Audio Manager

History Manager

Settings Manager

Logging System

4. Startup Sequence

Application startup follows this order.

```
launch.bat
```

```
↓
```

```
Bootstrap
```

```
↓
```

```
Environment Validation
```

```
↓
```

```
Virtual Environment
```

```
↓
```

```
Dependency Validation
```

```
↓
```

```
Folder Validation
```

```
↓
```

```
Hugging Face Cache Validation
```

```
↓
```

```
Model Validation
```

```
↓
```

Model Warmup

↓

SpeechStudio UI

Every stage must complete successfully before continuing.

5. Portable Design

The application shall never require installation.

The complete application must operate from a single directory.

The project directory contains every resource required for execution.

No Registry entries.

No installer.

No AppData usage.

No user profile storage.

No system wide configuration.

6. Project Directory

Example structure

```
SpeechStudio/  
  
  .cache/  
  
  logs/  
  
  models/  
  
  outputs/  
  
  presets/  
  
  settings/
```

```
temp/  
voices/  
engine/  
ui/  
spec/  
research/  
launch.bat  
bootstrap.py  
SpeechStudio.py
```

All paths must be relative to the application root.

7. Bootstrap Responsibilities

Bootstrap is responsible for preparing the runtime environment.

Tasks include

Create virtual environment if missing

Install missing dependencies

Create required folders

Configure Hugging Face cache

Verify CUDA

Verify Python version

Download tokenizer if required

Download model if required

Validate downloaded files

Start SpeechStudio

Bootstrap shall never generate speech.

8. Runtime Lifecycle

The application follows this lifecycle.

Application Start

↓

Initialization

↓

Model Loading

↓

Ready

↓

Generation Loop

↓

Application Shutdown

The model remains loaded throughout the Ready and Generation Loop states.

9. Model Lifecycle

The tokenizer is loaded once.

The model is loaded once.

The model remains resident until shutdown.

Multiple model instances are prohibited.

10. Configuration Storage

SpeechStudio stores configuration inside the project directory.

Configuration includes

Application settings

Recent files

Voice profiles

Presets

Window layout

History metadata

Configuration files shall use JSON format.

11. Voice Storage

Voice profiles reside inside

voices/

Each profile contains

Reference audio

Reference transcript

Metadata

Optional preview image

Each profile exists independently.

12. Output Storage

Generated files are written to

outputs/

Each generation stores

Generated audio

Metadata

Generation parameters

Timestamp

Prompt information

13. Hugging Face Cache

SpeechStudio shall configure a local Hugging Face cache.

The cache resides inside

`.cache/huggingface/`

The application shall never rely on the default user cache location.

14. Dependency Management

Dependencies are installed from

`requirements.txt`

SpeechStudio shall install missing packages only.

Installed packages are reused on subsequent launches.

15. Logging

Logs are stored inside

`logs/`

Log files include

Application

Bootstrap

Generation

Errors

Performance

Every exception shall be logged.

16. Error Recovery

Recoverable failures include

Missing model

Missing tokenizer

Missing folder

Missing dependency

Corrupted cache

Missing output folder

When possible

SpeechStudio shall repair the problem automatically.

If automatic recovery is impossible

A clear error message shall be presented.

17. Performance Goals

The model remains loaded.

Tokenizer remains loaded.

Reference voices remain cached while active.

Temporary objects shall be released after generation.

The UI shall remain responsive.

18. Thread Model

UI Thread

User interaction only.

Generation Thread

Speech synthesis.

Playback Thread

Audio playback.

Background Tasks

Bootstrap operations.

File indexing.

History updates.

No long running operation shall execute on the UI thread.

19. Shutdown Sequence

Shutdown order

Stop playback

Finish active generation if possible

Flush logs

Save settings

Release model

Release tokenizer

Close application

20. Offline Operation

After initial setup

SpeechStudio shall operate without internet access.

Internet is only required for

Initial dependency installation

Initial model download

21. System Constraints

Supported operating system

Windows

Supported backend

Transformers

Supported model

Higgs Audio V3 TTS

GPU acceleration

CUDA

Python runtime

Project local virtual environment

22. Success Criteria

SpeechStudio is considered correctly configured when

The application starts from launch.bat

The virtual environment is managed automatically

The required model is available

The tokenizer is available

The user reaches the main interface without opening a terminal

Speech generation can begin immediately